Copyright All rights reserved

03. 자료형과 변수

Heesung Oh
https://github.com/3dapi

3.1 데이터 표현과 메모리 단위

Heesung Oh
Copyright All rights reserved

- 컴퓨터 데이터 표현
  - 2진수(bit, 0과 1)
- 사람의 이해를 위해
  - 8진수, 10진수, 16진수 사용
- 2진수와 16진수
  - 컴퓨터 내부 데이터 확인에 유용한 표기법

3.1.1 데이터 표현 방식

Heesung Oh
Copyright All rights reserved

진법	설명	접두사	예시
10진수 (Decimal)	사람이 사용	없음	123, 2048
2진수 (Binary)	컴퓨터 내부 표현과 동일 (C23 이후 지원)	0b	0b101101
8진수 (Octal)	3bit 단위 표현. 권한 설정(owner, group, other)에서 사용	0	0755, 012
16진수 (Hexadecimal)	4bit = 1자리 대응 메모리·주소·색상 값 표현에 사용	0x	0xFF00CCAA

3.1.1 데이터 표현 방식

Heesung Oh
Copyright All rights reserved

- 같은 값을 다르게 표현하는 예
  - 42 (10진수)
  - 0b00101010 (2진수)
  - 052 (8진수)
  - 0x2A (16진수)

3.1.2 비트(bit)와 바이트(byte)

Heesung Oh
Copyright All rights reserved

- 비트(bit)
  - binary digit의 약자
  - 0 또는 1을 표현하는 최소 정보 단위
- 바이트(byte)
  - 8bit로 구성된 기본 저장 단위
  - C언어, 시스템 데이터의 기본 단위
- 단위 관계
  - 1 Byte = 8 bit
  - 1 KB(kilo byte) = 1,024 Byte
  - 1 MB(mega byte) = 1,024 KB
  - 1 GB(giga byte) = 1,024 MB
  - 1 TB(tera byte) = 1,024 GB
  - 1 PB(peta byte) = 1,024 TB
  - 1 EB(exa byte) = 1,024 PB
  - 1 ZB(zetta byte) = 1,024 EB

3.1.3 ASCII 코드

Heesung Oh
Copyright All rights reserved

- ASCII (American Standard Code for Information Interchange)
  - 문자(영문자, 숫자, 기호)를 숫자로 표현하는 문자 인코딩 표준

- 특징
  - 7비트(0~127) 코드 사용
  - 영문 대소문자, 숫자, 기호, 제어 문자 포함

- 구성
  - 제어: 0(NULL)~31, 127(Delete)
  - 기호: 32 ~ 47, 58 ~ 64, 91 ~ 96, 123 ~ 126
  - 숫자: 48(0) ~ 57(9)
  - 영문자
    - 대문자: 65(A) ~ 90(Z)
    - 소문자: 97(a) ~ 122(z)

Heesung Oh
Copyright All rights reserved

- 제어 문자 출력

```
#include <stdio.h>

int main(void)
{
    printf("\\n : %d\n", '\n');
    printf("\\t : %d\n", '\t');
    printf("\\0 : %d\n", '\0');

    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- 숫자에 대한 ASCII 코드

```
#include <stdio.h>

int main(void)
{
    printf("0 : %d\n", '0');
    printf("1 : %d\n", '1');
    printf("2 : %d\n", '2');
    printf("3 : %d\n", '3');
    printf("4 : %d\n", '4');
    printf("5 : %d\n", '5');
    printf("6 : %d\n", '6');
    printf("7 : %d\n", '7');
    printf("8 : %d\n", '8');
    printf("9 : %d\n", '9');

    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- 영문자에 대한 ASCII 코드

```c
#include <stdio.h>

int main(void)
{
    printf("A : %d\n", 'A');
    printf("B : %d\n", 'B');
    printf("Z : %d\n", 'Z');

    printf("a : %d\n", 'a');
    printf("b : %d\n", 'b');
    printf("z : %d\n", 'z');

    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- 영문자에 대한 ASCII 코드

```
#include <stdio.h>

int main(void)
{
    printf("! : %d\n", '!');
    printf("# : %d\n", '#');
    printf("@ : %d\n", '@');
    printf("[ : %d\n", '[');
    printf("{ : %d\n", '{');

    return 0;
}
```

3.1.4 정수 오버플로와 언더플로

Heesung Oh
Copyright All rights reserved

- 정수형은 정해진 비트 수로 값을 표현
- 표현 범위를 초과하면 값이 순환(wrap-around)

- 오버플로(Overflow)
  - 정수형 값이 최대 범위를 초과할 때 발생, 값이 가장 작은 값으로 순환
- 언더플로(Underflow)
  - 정수형 값이 최소 범위보다 작아질 때 발생, 값이 가장 큰 값으로 순환

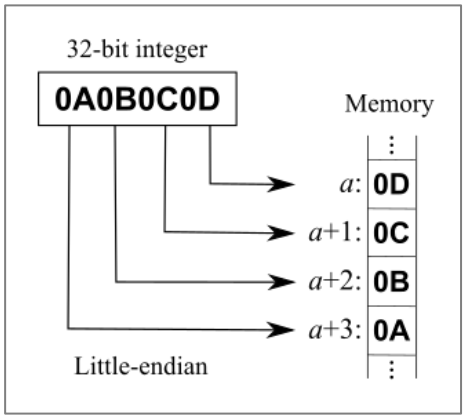
```
unsigned char n1 = 255;
n1++;          // 0으로 돌아감 (오버플로)

signed char n2 = 127;
n2++;
```

3.1.5 엔디안(Endianness)

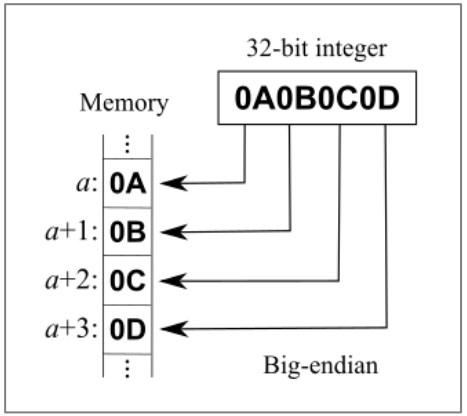
Heesung Oh
Copyright All rights reserved

- 엔디안
  - 멀티바이트 자료형(int, float 등)을 메모리에 저장하거나 읽을 때 바이트 배열 순서

- Little Endian
  - 하위 바이트 → 낮은 주소
    - 예: 0x12345678 → 78 56 34 12
  - 대부분의 CPU(x86, ARM) 사용



- Big Endian
  - 상위 바이트 → 낮은 주소
    - 예: 0x12345678 → 12 34 56 78
  - 네트워크 통신 표준 (Network Byte Order)



3.2 자료와 자료형

Heesung Oh
Copyright All rights reserved

- 자료 (Data)
  - 컴퓨터 메모리에 저장된 이진 데이터의 집합
  - 0과 1로 이루어진 bit group
  - 단순한 저장 상태

- 자료형 (Data Type)
  - bit group(저장 된 값)을 어떻게 해석할지 결정하는 규칙
  - 같은 bit group이라도 자료형에 따라 의미가 달라짐

- 자료형 예: 1078530011
  - int → 1078530011
  - float → 3.141593
  - pointer → 0x40490FDB (32bit) → 메모리 주소 0x40490FDB
  - 같은 bit group이라도 자료형에 따라 의미가 달라짐

3.3 변수와 변수의 6가지 속성

Heesung Oh
Copyright All rights reserved

- 변수 (Variable)
  - 값을 저장하기 위해 이름이 붙은 메모리 공간
  - 특정 자료형(Type)을 기반으로 생성
  - 프로그램 실행 중 값을 읽고 변경 가능
  - 프로그램의 상태(state) 표현

- 상수(Constant)
  - 값이 변하지 않는 데이터

- 변수의 6가지 속성
  - 이름, 형식, 값, 저장 위치, 범위, 수명

3.3.1 변수의 6가지 속성

Heesung Oh
Copyright All rights reserved

- 1. 이름 (Name)
  - 변수의 식별자(identifier)
  - 메모리 공간을 참조하기 위한 이름
  - 규칙
    - 알파벳, 숫자, _ 사용
    - 숫자로 시작 불가
    - 대소문자 구분
    - 키워드 사용 불가

- 2. 형식 (Type)
  - 변수의 자료형(data type)
  - 메모리 크기 / 해석 방식 / 연산 규칙 결정
  - 예
    - int → 정수
    - float → 실수
    - char → 문자

3.3.1 변수의 6가지 속성

Heesung Oh
Copyright All rights reserved

- 3. 값 (Value)
  - 자료형을 기준으로 해석된 의미 있는 데이터
  - 예
    - int n = 65; → 숫자 65
    - char c = 65; → 문자 'A'

- 4. 저장 위치 (Storage Location)
  - 변수가 저장되는 메모리 영역
  - Stack
    - 지역 변수
    - 함수 매개변수
  - Heap
    - malloc(), calloc(), realloc() 등 동적으로 할당한 메모리
  - Data Segment
    - 전역 변수
    - static 변수

3.3.1 변수의 6가지 속성

Heesung Oh
Copyright All rights reserved

- 5. 범위 (Scope)
  - 변수에 접근 가능한 코드 영역
  - Block Scope → { } 내부
  - File Scope → 전역 변수
  - Function Scope → goto 레이블

- 6. 수명 (Lifetime)
  - 변수가 메모리에 존재하는 기간
  - 자동(Automatic) 변수
    - 코드 블록 시작({) ~ 블록 종료(})
  - 정적(Static) 변수
    - 프로그램 시작 ~ 종료
  - 동적(Dynamic) 변수
    - malloc/free로 직접 관리

3.4 C언어 키워드

Heesung Oh
Copyright All rights reserved

- C 언어 표준 키워드 32개
  - auto break case char const continue default do double
  - else enum extern float for goto if int long register
  - restrict return short signed sizeof
  - static struct switch typedef union unsigned void volatile
- C11 에서 추가된 키워드
  - _Alignas _Alignof _Atomic _Bool _Generic
  - _Noreturn _Static_assert _Thread_local
- C23 에서 추가된 키워드
  - bool true false alignas alignof thread_local

3.5 변수 선언과 코드 블록에서의 위치

Heesung Oh
Copyright All rights reserved

- **변수 선언 (Variable Declaration)**
  - 데이터를 저장할 메모리 공간의 이름과 자료형을 지정하는 과정
  - 변수는 코드 블록 { } 내부에서 선언
  - C99 이후 사용 위치와 가까운 곳에서 선언하는 방식이 일반적

- **변수 선언 기본 형태**

```
자료형  변수이름;
```

- **예**

```
int age;
float height;
int level;
```

3.5 변수 선언과 코드 블록에서의 위치

Heesung Oh
Copyright All rights reserved

- **초기화하지 않은 지역 변수 (Uninitialized Value)**
  - 선언만 하고 초기화하지 않으면 메모리 값이 불확정(Indeterminate Value)
  - 이전 메모리 값이 남아 있을 수 있음
  - 실행 환경마다 값이 달라질 수 있음
  - Visual Studio Debug 모드
    - 미초기화 메모리에 0xCC 패턴을 채우기도 함
  - ※ 쓰레기 값(garbage value) 으로 채운다고 설명하는 것은 잘못된 표현임

- **선언과 동시에 초기화**

```
자료형 변수이름 = 초기 값;

// 같은 자료형인 경우 콤마 연산자를 사용해서 초기화
자료형 변수이름 = 초기 값, 변수이름2 = 초기 값;
```

```
int level=100;
int age = 0, money = 10000;
float width= 0.0f height = 0.0f;
bool isAlive = false; // C23 이후 2진수 허용
```

3.5.1 C90 스타일

Heesung Oh
Copyright All rights reserved

- 초기 C 표준 C90에서는 변수 선언을 코드 블록의 시작 부분에서만 허용
- 실행 문장이 시작된 이후에는 새로운 변수 선언 불가

```
int main(void)
{
    int a;   // C90: 블록 시작 부분에서 선언해야 함
    int b = 10;

    printf("%d\n", b);
}
```

- 블록 중간 선언은 C90에서 허용되지 않음

```
int main(void)
{
    printf("start\n");
    int x = 10;   // C90: 오류 발생
}
```

3.5.2 C99 이후

Heesung Oh
Copyright All rights reserved

- C99 이후(C99, C11, C17, C23) 변수 선언을 코드 블록의 중간에서도 허용
- 필요한 위치에서 변수를 바로 선언 가능
- 현대 C 언어에서는 이 방식을 일반적으로 사용

```
int main(void)
{
    printf("start\n");

    int hp = 100;      // C99 이후: 중간 선언 가능
    printf("%d\n", hp);
}
```

3.5.3 권장되는 선언 방식(최신 C 스타일)

Heesung Oh
Copyright All rights reserved

- Visual Studio(MSVC)도 현재 C99 스타일 변수 선언을 대부분 지원
- 사용되는 위치와 가까운 곳에서 변수 선언 권장
- 변수의 의미가 명확해지고 가독성 향상
- 블록 앞부분에 불필요하게 변수를 모아 두지 않는 것이 좋음

- 반복문에서 변수 선언
  - for 문의 초기식에서 변수 선언 권장

```
for(int i=0, j=0; i<10; ++i, ++j)
{
    printf("%d\n", i, j);
}
```

3.6 기본 자료형

Heesung Oh
Copyright All rights reserved

- 기본 자료형
  - CPU의 기본 연산 단위(32bit / 64bit)를 기반으로 정의된 기초 자료형
  - 정수, 실수, 주소, 논리 값 등의 기본 데이터를 표현

- 특징
  - 다양한 정수 크기(char, short, int, long) 제공
  - 메모리 효율성과 데이터 표현의 유연성 제공

※ 기본 자료형 = CPU 데이터 폭을 기반으로 데이터 표현의 최소 단위

3.7 논리형(boolean types)

Heesung Oh
Copyright All rights reserved

- 논리형 (Boolean Type)
  - 참(true) / 거짓(false) 을 표현하는 자료형
- C90
  - 별도의 논리형 없음
  - 0 → false, 0이 아닌 값 → true
  - int를 논리값처럼 사용
  - 매크로 기반
    - #define TRUE 1
    - #define FALSE 0
- C99
  - 〈stdbool.h〉 헤더 도입
  - bool, true, false 사용 가능
- C23
  - bool, true, false가 정식 키워드로 추가
  - 헤더 없이 사용 가능

3.7 논리형 (boolean types)

Heesung Oh
Copyright All rights reserved

- 〈예: boolean〉

```
#include <stdbool.h>
#include <stdio.h>

int main(void)
{
    bool isReady = true, isActive=false;
    printf("value isReady: %d\n", isReady);
    printf("value isActive: %d\n", isActive);

    return 0;
}
```

3.8 정수형 (Integer Types)

Heesung Oh
Copyright All rights reserved

- **C 언어 정수형과 표준 최소 크기 / 현대 시스템 일반 크기**
  - **C 표준은 최소 크기만 보장, 실제 크기는 시스템과 컴파일러에 따라 달라짐**

- **정수형 종류**

자료형	C 표준 최소 크기	현대 시스템 일반 크기
char	8 bit	1 byte
short	16 bit	2 byte
int	16 bit	4 byte
long	32 bit	4 또는 8 byte
long long	64 bit	8 byte

3.8.1 정수형 데이터 범위

Heesung Oh
Copyright All rights reserved

- **정수형 signed / unsigned 범위**

	signed 범위	unsigned 범위
1 byte	-128 ~ 127	0 ~ 255
2 byte	-32,768 ~ 32,767	0 ~ 65,535
4 byte	-2,147,483,648 ~ 2,147,483,647	0 ~ 4,294,967,295
8 byte	-9.22e18 ~ 9.22e18	0 ~ 1.84e19

3.8.2 정수형 출력

Heesung Oh
Copyright All rights reserved

- 〈예: 정수형〉

```c
#include <stdio.h>

int main(void)
{
    char ch0='A', ch1=66;
    int number1, number2;
    int number3=1111;
    int number4=2222;
    printf("ch0: %d, ch0: %c \n", ch0, ch0);
    printf("ch1: %d, ch1: %c \n", ch1, ch1);

    // 최신 컴파일러에서는 초기화 안하면 컴파일 에러가 발생하나,
    // 오래된 컴파일러는 문제 없이 넘어감
    printf("number1: %d, number2: %d \n",number1,number2);

    number1=6666;
    number2=7777;

    printf("number1: %d, number2: %d \n",number1,number2);
    printf("number3: %d, number4: %d \n",number3,number4);

    return 0;
}
```

3.8.3 부호 있는 정수의 표현 방식

Heesung Oh
Copyright All rights reserved

- 부호: MSB(Most Significant Bit)
  - 양수: 0, 음수: 1
- 음수
  - 2의 보수를 이용
- 예: -5

```
00000101     +5
  ↓
11111010     1의 보수
  ↓
11111011     +1을 추가해서 2의 보수로 변경
```

```
<검증>
   00000101  (+5)
  +11111011  (-5)
  ----------
  100000000
  ↑
  올림(carry) 값 버림
```

3.8.4 데이터 모델(Data Model)

Heesung Oh
Copyright All rights reserved

- 데이터 모델 (Data Model)
  - int, long, pointer의 크기를 정의하는 규약
- long 크기 차이
  - 정수형 중 플랫폼 차이가 가장 큰 자료형은 long
  - Android/iOS/macOS/Linux: long = 8바이트
  - Windows MSVC: long = 4바이트

- 데이터 모델

모델	int	long	pointer	사용 환경
ILP32	32bit	32bit	32bit	임베디드, Win32, 32bit 모바일
LP64	32bit	64bit	64bit	모바일·64bit: Android, iOS macOS, Linux 64bit
LLP64	32bit	32bit	64bit	Windows 64bit

3.8.5 고정폭 정수형(stdint.h)

Heesung Oh
Copyright All rights reserved

- **정확한 크기의 정수**
  - #include ⟨stdint.h⟩ 추가

타입	크기	타입	크기
int8_t	8bit	**uint8_t**	8bit
int16_t	16bit	**uint16_t**	16bit
int32_t	32bit	**uint32_t**	32bit
int64_t	64bit	**uint64_t**	64bit

3.9 실수형(Floating Types)

Heesung Oh
Copyright All rights reserved

- **실수형**
  - 실수를 표현하는 자료형
  - 부동소수점(Floating-Point) 방식 사용
  - IEEE 754 표준 기반
  - 실수 계산 → FPU (Floating Point Unit)

- **실수형의 종류와 크기**
  - Windows·Linux·macOS는 double을 64비트로 동일
  - long double은 플랫폼마다 80비트/64비트/128비트 등 다양

자료형	크기	설명
float	32비트(4바이트)	단정도(single precision)
double	64비트(8바이트)	배정도(double precision)
long double	구현 의존	확장 정밀도(extended precision)

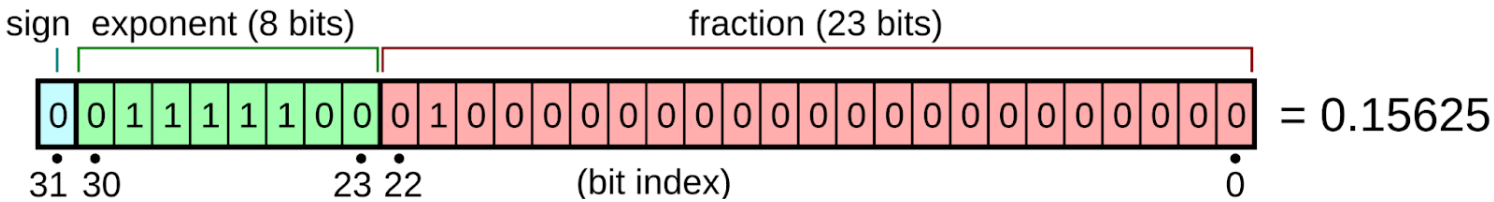
3.9.2 IEEE 754 부동소수점 표준 개요

Heesung Oh
Copyright All rights reserved

- **IEEE 754 실수 표현**

$$N = (-1)^{sign} \times 2^{(exponent-127)} \times (1 + \frac{mantissa_bits}{2^{23}})$$

- **32bit 부동 소수점 메모리 구조**



- **구성 요소**

구성 요소	float	double	설명
부호(sign)	1비트	1비트	0 = 양수, 1 = 음수
지수(exponent)	8비트	11비트	2의 지수 승
가수(fraction, mantissa)	23비트	52비트	소수 부분

3.9.3 float/double 정밀도와 표현 범위

Heesung Oh
Copyright All rights reserved

항목	float (단정도, 32비트)	double (배정도, 64비트)
총 비트 수	32비트	64비트
정밀도	약 6~7자리	약 15~16자리
지수 범위	−126 ~ +127	−1022 ~ +1023
표현 가능한 양수 최소값	약 1.18 × 10^−38	약 2.23 × 10^−308
표현 가능한 양수 최대값	약 3.40 × 10^38	약 1.79 × 10^308
특수값	±0, ±∞, NaN	±0, ±∞, NaN

3.9.4 부동소수점의 특수값 과 선택

Heesung Oh
Copyright All rights reserved

- **부동소수점의 특수값**

값	의미
+0 / −0	부호가 있는 0
+∞ / −∞	오버플로우 또는 1/0.0 등
NaN	연산 불능(Not a Number), 0/0, 음수의 제곱근 등

- **float, double 선택 기준**

항목	float	double
메모리	작다(4B)	크다(8B)
연산 속도	빠름(특히 GPU·모바일)	float보다 느림
정밀도	약 7자리	약 16자리
사용 용도	실시간 게임, GPU 연산	물리 시뮬레이션, 금융, 과학 계산, 시간 누적

3.9.6 부동 소수점 주의 사항

Heesung Oh
Copyright All rights reserved

- 1) NaN은 알 수 없음 상태. 어떤 값과도 불 일치

```
if (x == x)  // x가 NaN이면 false
```

- 2) 실수형 비교 시 주의사항
  - == 비교는 대부분 실패

```
// 잘못된 예
if (a == b) { ... }

// 입실론 값을  사용해서 판단
#include <math.h>
//...
if (fabs(a - b) < 1e-6) { ... }
```

3.9.6 부동 소수점 주의 사항

Heesung Oh
Copyright All rights reserved

- 3) 부동소수점 연산 규칙의 특징
  - 오차 누적 순서가 달라지기 때문에 덧셈/뺄셈은 교환법칙이 성립하지 않음
  - (a + b) + c ≠ a + (b + c)

```
double a = 1e16;
double b = -1e16;
double c = 1.;
printf("%f\n", (a + b) + c); // 1

// 매우 큰 값과 작은 값을 더할 때
// 작은 값이 반올림으로 사라질 수 있음
printf("%f\n", a + (b + c)); // 0

// 부동소수점 반복 연산 오차
// 0.1은 이진 부동소수점으로 정확한 표현이 안됨.
// 반복 덧셈을 하면 오차가 누적
float stored = 0.0F;
for(int i=0; i<100; i++)
    stored += 0.1F;
printf("%f \n", stored);
```

3.9.6 부동 소수점 주의 사항

Heesung Oh
Copyright All rights reserved

- 4) 정수형과 혼합 연산할 때 주의
  - 실수형 정밀도 제한 → 일정 범위 정수만 정확히 표현
  - 예) float → 약 7 자리까지만 정확히 표현

```
int i = 1234567890;
float f = i;   // int → float 변환 (정밀도 손실 가능)

printf("int   : %d\n", i);
printf("float: %.0f\n", f);  // 반올림된 값 출력
```

3.10 enum 형 (열거형)

Heesung Oh
Copyright All rights reserved

- enum
  - 사용자가 정의한 의미 있는 이름을 가진 정수 상수들의 집합

```
enum Direction { LEFT, RIGHT, UP, DOWN };

enum GAME_STATE { NONE, BEGIN, PLAY, END };
```

3.11 배열 형 (Array)

Heesung Oh
Copyright All rights reserved

- 배열형
  - 기본 자료형 기반 파생형 자료구조
  - 동일한 자료형의 값을 연속된 메모리 공간에 저장하는 자료 구조

- 특징
  - 하나의 이름으로 여러 데이터를 관리
  - 같은 자료형만 저장 가능
  - 다수의 데이터를 처리할 때 유용

```
int scores[5];
int items[128] = {0};
```

3.12 포인터형 (Pointer)

Heesung Oh
Copyright All rights reserved

- **포인터**
  - 파생형 자료구조
  - 메모리의 "주소(address)" 를 저장하는 자료형
  - 값이 저장된 메모리 위치를 가리킴

- **void* (범용 포인터)**
  - 어떤 자료형의 주소도 저장 가능
  - 자료형이 정해지지 않아 직접 값 접근 불가
  - 사용 시 적절한 포인터 타입으로 캐스팅 필요

```
int* p = &x;
```

```
void* vp = &x;            // 어떤 자료형의 주소 든 저장 가능
int value = *(int*)vp;    // void* → int* 로 캐스팅 후 값 읽기
```

3.13 구조체(struct)

Heesung Oh
Copyright All rights reserved

- 구조체 (struct)
  - 서로 다른 자료형을 하나의 묶음으로 구성하는 사용자 정의 자료형

- 특징
  - 여러 데이터를 하나의 단위로 관리
  - 각 멤버는 독립된 메모리 공간을 가짐
  - 서로 다른 자료형을 하나로 결합 가능

```
struct Player
{
    int hp;
    float speed;
};
```

3.14 공용체(union)

Heesung Oh
Copyright All rights reserved

- 공용체
  - 여러 자료형이 하나의 메모리 공간을 공유하는 사용자 정의 자료형

- 특징
  - 모든 멤버가 같은 메모리 영역 사용
  - 동시에 하나의 멤버만 유효
  - 크기 → 가장 큰 멤버 기준

```
union Data {
    int n;
    float f;
};
```

3.15 typedef (형 별칭)

Heesung Oh
Copyright All rights reserved

- typedef (형 별칭)
  - 기존 자료형에 새로운 이름(별칭, alias) 을 부여하는 기능

- 특징
  - 새로운 자료형을 만드는 것이 아니라 기존 자료형의 이름을 재정의
  - 복잡한 선언을 읽기 쉽고 관리하기 쉽게 만듦

```
typedef unsigned int UINT;
typedef double Money;
typedef char CHUNK[128];
typedef struct Player *PPLAYER;
```

3.16 상수(Constant)

Heesung Oh
Copyright All rights reserved

- 상수(Constant)
  - 프로그램 실행 동안 값이 변하지 않는 데이터

- 특징
  - 읽기 전용(Read-only) 값
  - 값 변경 방지 → 코드 안정성 향상

- 상수의 종류
  - 기호 상수 (Symbolic Constant)
  - 리터럴 상수 (Literal Constant)

3.16.1 기호(Symbolic) 상수

Heesung Oh
Copyright All rights reserved

- 기호 상수 (Symbolic Constant)
  - 값 대신 의미 있는 이름을 사용하는 상수
- 형태
  - const 상수, 매크로 상수 (#define)
- const 상수
  - 읽기 전용 변수. 선언 시 초기값 필요. 이후 값 변경 시 컴파일 오류

```
const int MAX_WIDTH = 32;
```

```
void PrintMessage(const char* msg)
{
    // msg를 변경할 수 없음
}
```

- 매크로 상수
  - 전처리기(preprocessor) 단계에서 처리
  - 컴파일 전에 단순 치환

```
#define MAX_ITEM 128
#define MAX_LENGTH 32
```

3.16.2 리터럴(Literal)

Heesung Oh
Copyright All rights reserved

- 리터럴
  - 코드에 직접 작성된 값 자체
- 종류
  - 정수 리터럴, 실수 리터럴, 문자 리터럴, 문자열 리터럴
- 기본 규칙
  - 소수점 없음 → 정수 리터럴
  - 소수점 있음 → double 리터럴
- 지수형 리터럴
  - e 또는 E 사용
    - 1.23e3 → 1.23×10^3
    - 5.0e-2 → 5.0×10

Heesung Oh
Copyright All rights reserved

리터럴 접두사(Prefix)·접미사(Suffix)

종류	형태	의미	사용 예
접두사	0b	2진수(C23 이후)	0b1011
	0	8진수	077
	0x	16진수	0xFF
접미사	u / U	unsigned int	100U
	l / L	long	100L
	ul / UL	unsigned long	100UL
	ll / LL	long long	123LL
	(없음)	소수점 없음: signed int	256
		소수점 있음: double	3.14
	f / F	float	3.14f, 3.14F
	l / L	long double	1.23L

3.16.2 리터럴(Literal)

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>
int main(void)
{
    float  a = 1.2345678e3F; // 1.23 × 10^3
    double b = 9.0e-4; // 9.0 × 10^-4

    // 디폴트 출력: 1.234568e+03, 1.234568E+03,
    // 9.000000e-04, 9.000000E-04
    printf("%e, %E, %e, %E\n", a, a, b, b);

    // 소수점 자리수 제한 1.235e+03, 1.235E+03,
    // 9.0000e-04, 9.0000E-04
    printf("%.3e, %.3E, %.4e, %.4E\n", a, a, b, b);

    return 0;
}
```

3.16.2 리터럴(Literal)

Heesung Oh
Copyright All rights reserved

```
// 정수형 리터럴
1234    // int 10진수
0xFF    // int 16진수
0775    // int  8진수
0b1011  // boolean C23 이후 2진수 허용

// 실수형 리터럴
3.        // double 리터럴 3.0과 같다
3.0       // double 리터럴 3. 과 같다
3.14      // double 리터럴 지수 표현
1.0e-3    // double 리터럴
1.23L     // long double 리터럴
3.14F     // float 리터럴
2.217f    // float 리터럴

// 문자, 문자열 리터럴
'A'       // 문자 리터럴
"hello"   // 읽기 전용 문자열 리터럴
"ab\nxy"  // 읽기 전용 문자열 리터럴
```

3.16.2 리터럴(Literal)

Heesung Oh
Copyright All rights reserved

- 리터럴의 자료형 해석
  - 리터럴은 특정 자료형으로 자동 해석

- 문자열 리터럴
  - 읽기 전용 메모리(RODATA)에 저장
  - 수정 불가
  - 내부적으로 널 문자('₩0') 포함

```
100 → int
3.14 → double
3.14F → float
'A' → char
"hello" → 읽기 전용 메모리에 저장된 문자열
```

3.16.3 NULL 포인터, NULL 문자

Heesung Oh
Copyright All rights reserved

- 0/NULL 포인터 / NULL 문자
  - 값 0은 문맥에 따라 다른 의미로 해석

표현	의미
0	정수 0
NULL	아무 것도 가리키지 않는 포인터
'\0'	문자열 종료 문자

- NULL 포인터

```
int* p = NULL;
```

- NULL 문자

```
char s[] = "Hello";
// s[] = {'H', 'e', 'l', 'l', 'o', '\0'}; 와 동일
```

3.17 중괄호 {}를 이용한 초기화

Heesung Oh
Copyright All rights reserved

- C 언어 변수 초기화 방식
  - 스칼라 타입 (Scalar Type), 집계 타입 (Aggregate Type)
  - C99: 스칼라 타입에 중괄호 {} 초기화 허용

- 스칼라 타입 (Scalar Type)
  - 단일 값을 가지는 자료형

```
int     a = {100};
int     a2= {100, 200}; // 오류: 단일 값만 허용
int     b = {100}, c = {200};
double d = {3.14};
char    c = {'A'};
int*    p = {NULL};
```

3.17 중괄호 {}를 이용한 초기화

Heesung Oh
Copyright All rights reserved

- 집계 타입 (Aggregate Type)
  - 여러 값을 묶어 저장하는 자료형

```
// 배열
int arr[5] = {1, 2, 3};
// 구조체
struct Point { int x; int y; int z; };
struct Point p = { 10, 20 };
// 공용체
union Data { int i; float f; };
union Data d = { 10 };   // i = 10으로 초기화
```

3.17.3 Zero Initialization (자동 0 초기화)

Heesung Oh
Copyright All rights reserved

- Zero Initialization 규칙
  - 초기화 리스트 {} 에서 모든 값을 제공하지 않으면 남은 요소는 자동으로 0으로 초기화
  - Zero Initialization은 중괄호 초기화가 사용될 때만 적용
  - 지역 변수의 자동 초기화와 혼동해서는 안됨

타입	초기값
정수형	0
실수형	0.0
문자형(char)	'₩0'
논리형(_Bool)	0 (false)
포인터	NULL
구조체 멤버	각 멤버 타입에 따라 0
배열 요소	0
공용체	초기화된 멤버만 의미를 가짐

3.17.3 Zero Initialization (자동 0 초기화)

Heesung Oh
Copyright All rights reserved

- 배열 예

```
int arr[5] = {1, 2};
// 결과: [1, 2, 0, 0, 0]
```

- 구조체 예

```
struct Point p = { 10, 20 };
// 결과: p.x = 10, p.y = 20, p.z = 0
```

- 중첩 구조체

```
struct Pos { float x, y; };
struct Player {
    int hp;
    struct Pos pos;
};

struct Player p = { 100 };
//결과: hp = 100, pos.x = 0.0f, pos.y = 0.0f
```

3.18 형 변환

Heesung Oh
Copyright All rights reserved

- 형 변환
  - 형 변환은 한 자료형의 값을 다른 자료형으로 바꾸는 과정
  - 암시적 형 변환, 명시적 형 변환
- 암시적 형 변환 (Implicit Conversion)
  - 컴파일러가 자동으로 수행하는 형 변환
- 명시적 형 변환 (Explicit Conversion)
  - 프로그래머가 타입 변환

3.18.1 형 변환 – 암시적 형 변환

Heesung Oh
Copyright All rights reserved

- 컴파일러가 자동으로 수행
  - 서로 다른 자료형이 연산에 사용될 때
  - 대입 연산에서 좌변과 우변의 자료형이 다를 때
  - 함수 호출 시 인자의 자료형이 매개변수 타입과 다를 때

- 정수 승격 (Integer Promotion)
  - 표현 범위가 더 큰 자료형으로 형 변환
  - 값의 손실 없이 수행

- 축소 변환(Narrowing Conversion)
  - 표현 범위가 더 작은 자료형으로 형 변환
  - 값의 손실 발생 가능, 컴파일러에서 경고 발생 가능 (Narrowing warning)

3.18.1 형 변환 – 암시적 형 변환

Heesung Oh
Copyright All rights reserved

- 정수 승격(Integer Promotion)
  - 암시적 형 변환. int보다 작은 정수형이 연산에 사용되기 전 int형(또는 unsigned int형)으로 변환
  - 정수, _Bool, enum 등에도 적용
  - 정수 승격 이후 피연사자의 타입을 맞추기 위해 산술 형 변환 진행
  - 값이 int로 표현 가능하면 → int
    표현 범위를 초과하면 → unsigned int

```
// 정수 승격 signed → unsigned
char a = 10;
char b = 20;
printf("sizeof(a +10)  = %zu\n", sizeof(a + 10));
printf("sizeof(a + b)  = %zu\n", sizeof(a + b));

// 정수 승격 signed → unsigned
char c = -1;
unsigned int d = 1;
printf("sizeof(c + d)  = %zu, %d + %u = %u\n", sizeof(c+d), c, d, c+d);
```

3.18.1 형 변환 – 정수 승격 필요성

Heesung Oh
Copyright All rights reserved

- 하드웨어 효율성과 설계 단순화
  - CPU의 ALU는 일정한 워드 크기 기준으로 최적화
  - 모든 정수형 연산 장치를 따로 두지 않고 int 단위로 통합 처리
  - 작은 자료형도 레지스터 크기로 확장하여 연산
  - 규격화, 설계 단순화 → 비용 절감

- 연산 안정성과 결과 일관성
  - 연산 중 중간값 범위 확장으로 오버플로 위험 감소
  - 서로 다른 자료형 연산을 표준 크기로 변환하여 일관성 유지

- 데이터 확장 메커니즘
  - 현대 시스템에서도 int(보통 32비트)가 기본 연산 단위
  - signed → 부호 확장(Sign Extension)
    - 확장한 상위 비트를 1로 채워 부호 유지
  - unsigned → 제로 확장(Zero Extension)
    - 확장한 상위 비트를 0으로 채워 값 유지

3.18.1 형 변환 – 산술 형 변환

Heesung Oh
Copyright All rights reserved

- **산술 형 변환 (Usual Arithmetic Conversions)**
  - 다른 자료형의 피연산자 사이 연산에서 타입을 공통 타입(Common Type)으로 일치시키는 과정

```
int + float    → int가 float로 변환
int + double   → int가 double로 변환
float + double → float이 double로 변환

unsigned int + long long →
unsigned int가 long long 또는 unsigned long long으로 변환
```

3.18.1 형 변환 – 정수의 실수형 변환

Heesung Oh
Copyright All rights reserved

- Floating Type Conversion
  - 정수형과 실수형 연산에서 더 넓은 실수형으로 변환
  - 실수형 변수 = 정수 값 → 정수 값이 실수형 값으로 변환

```
int + float  → int가 float로 변환
int + double → int가 double로 변환
```

3.18.1 형 변환 – 축소 변환

Heesung Oh
Copyright All rights reserved

- 축소 변환 (Narrowing Conversion)
  - 정밀도, 자료형 크기의 축소로 값 손실
  - 컴파일러 경고 (Narrowing Warning)

```
float f = 1234567890;
// 정수 1234567890 → float 타입으로 변환

printf("%f\n", f);
// 1234567936.000000 (정밀도 손실 발생)
```

```
double d = 3.14;
int i = d;      // 3.14 → 3
```

3.18.2 형 변환 – 명시적 형 변환

Heesung Oh
Copyright All rights reserved

- 명시적 형 변환 (Explicit Casting)
  - 프로그래머가 직접 형 변환을 지정하는 방식
  - 큰 자료형을 작은 자료형으로 변환할 때 값 손실
  - 이에 대한 인지를 컴파일러에게 명확히 전달
  - 컴파일러 워닝(Warning) 축소

- 주의 사항
  - 값 손실 가능성
  - 형변환 위치에 따른 연산 순서

3.18.2 형 변환 – 명시적 형 변환

Heesung Oh
Copyright All rights reserved

- **캐스팅**

```
int a = 128;
long long b = a;
float f = 3.14159265358979323846 26;
int n   = f;
printf("b = %lld, %d \n",b,n);
```

경고 발생
(6,1): warning C4305: '초기화 중': 'double'에서 'float'(으)로 잘립니다.
(7,12): warning C4244: '초기화 중': 'float'에서 'int'(으)로 변환하면서 데이터가 손실될 수 있습니다.

- **명시적 형 변환**

```
float f = (float)3.14159265358979323846;   // double → float 명시적 변환
int   n = (int)f;                          // float → int 명시적 변환
```

3.18.2 형 변환 – 명시적 형 변환 주의 사항

Heesung Oh
Copyright All rights reserved

- 값 손실 가능성

```
long long big_val = 4294967296LL; // 2^32

int truncated = (int)big_val;
// 상위 비트 잘려 다른 값이 저장됨
```

- 형 변환 위치에 따른 연산 순서

```
int x1 = 128, x2 = 256;

double y1 = (double)x1 / x2;
double y2 = (double)(x1 / x2);
```

학습 정리

Heesung Oh
Copyright All rights reserved

- 데이터 표현과 메모리 단위
  - bit, byte, 2진수, 10진수, 16진수 표현 방법
- 자료와 자료형
  - 자료는 컴퓨터에 저장된 bit group
  - 자료를 해석하고 처리하는 방식을 규정하는 것이 자료형(Data Type)
- 변수와 기본 속성
  - 변수는 이름이 붙은 메모리 공간
  - 프로그램의 상태(state) 표현
  - 6가지 속성: 이름(Name), 자료형(Type), 값(Value), 저장 위치(Storage), 범위(Scope), 수명(Lifetime)
- C 언어 자료형
  - 기본 자료형: 논리형(boolean), 정수형(integer), 실수형(floating)
  - 파생형: 배열(array), 포인터(pointer)
  - 사용자 정의 자료형: 구조체(struct), 공용체(union), 열거형(enum)
  - 형 별칭: typedef

학습 정리

Heesung Oh
Copyright All rights reserved

- 상수(Constant)
  - 상수는 실행 중 값이 변하지 않는 데이터
  - const 키워드를 사용한 상수, #define, 리터럴(literal)

- 형 변환
  - 자료형의 값을 다른 자료형으로 변경하는 과정
  - 암시적 형 변환 (Implicit Conversion)
  - 명시적 형 변환 (Explicit Casting)

실습 과제

Heesung Oh
Copyright All rights reserved

- 과제 1: 게임 캐릭터의 상태 출력하기
  - 기사(Knight), 마법사(Mage), 성직자(Priest) 세 캐릭터를 준비하고 각 캐릭터의 HP, MP, EXP를 변수로 선언한 뒤 다음과 같은 형식으로 출력
  - 각 수치는 float 형을 사용

- 과제 2: 좋아하는 캐릭터 출력
  - 콘솔 화면에 32 × 32 크기의 문자 그래픽으로 자신이 좋아하는 게임 캐릭터 3개를 출력하는 프로그램을 작성
  - 각 캐릭터는 서로 다른 모습이 되도록 구성